Timer

Cấu trúc và cấu hình

- **RBPU – Bit cho phép điện trở kéo lên của PORTB**

0: Điện trở kéo lên (pull-up resistors) của PORTB bị vô hiệu hóa.
1: Các chân PORTB có thể kết nối với điện trở kéo lên.

- **INTEDG – Bit chọn cạnh ngắt**

0: Ngắt xảy ra ở cạnh lên (rising edge) của chân **INT** (0 → 1).
1: Ngắt xảy ra ở cạnh xuống (falling edge) của chân **INT** (1 → 0).

- **T0CS – Bit chọn nguồn xung cho TMR0**

0: Xung được đưa vào bộ đếm/bộ định thời TMR0 thông qua chân RA4.
1: Bộ định thời sử dụng xung nhịp nội bộ (Fosc/4).

- **T0SE – Bit chọn cạnh nguồn cho TMR0**

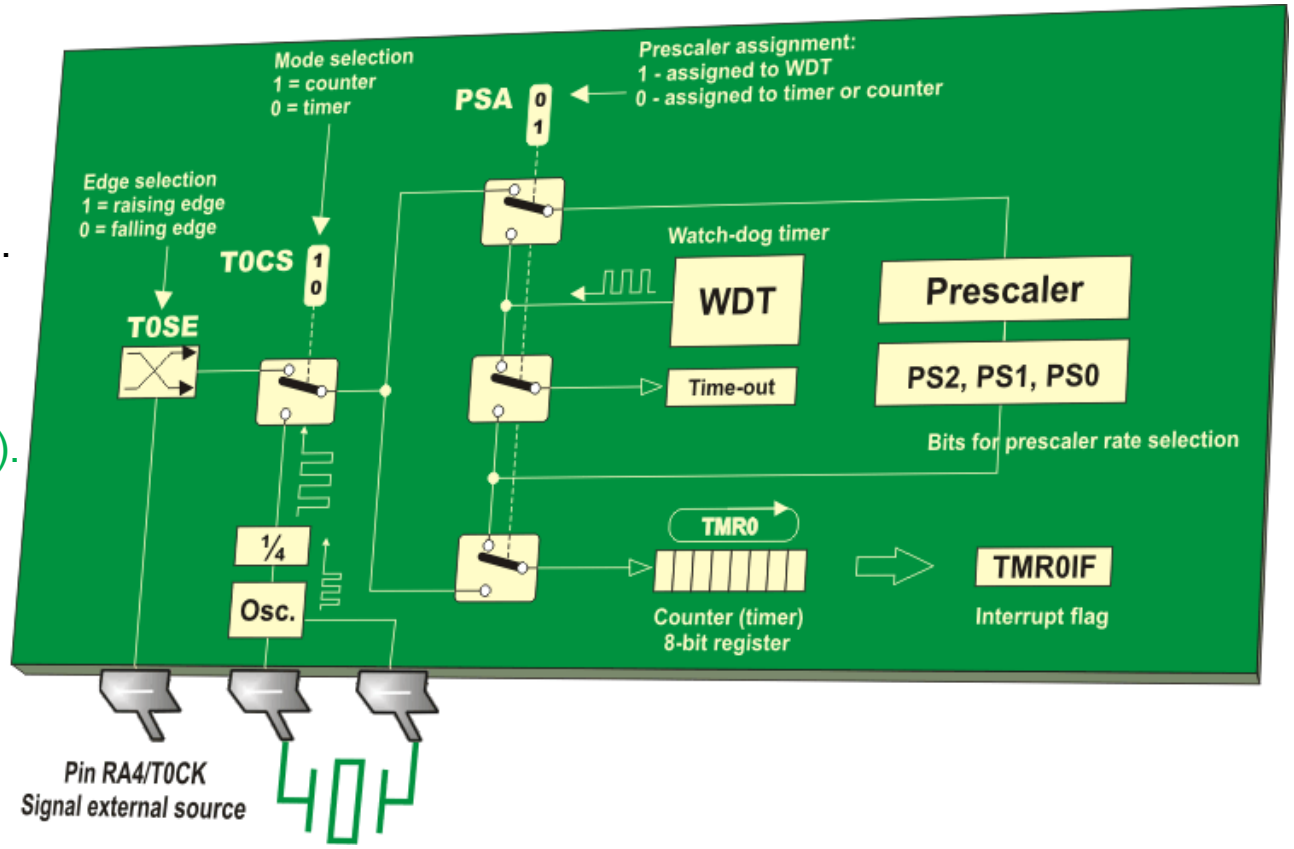
0: Tăng (increment) khi có chuyển mức từ cao xuống thấp (high-to-low) trên chân TMR0.
1: Tăng khi có chuyển mức từ thấp lên cao (low-to-high) trên chân TMR0.

- **PSA – Bit gán bộ chia tần (Prescaler Assignment)**

0: Bộ chia tần được gán cho WDT (Watchdog Timer).
1: Bộ chia tần được gán cho bộ đếm/bộ định thời TMR0.

- **PS2, PS1, PS0 – Các bit chọn tỷ lệ chia (Prescaler Rate Select)**

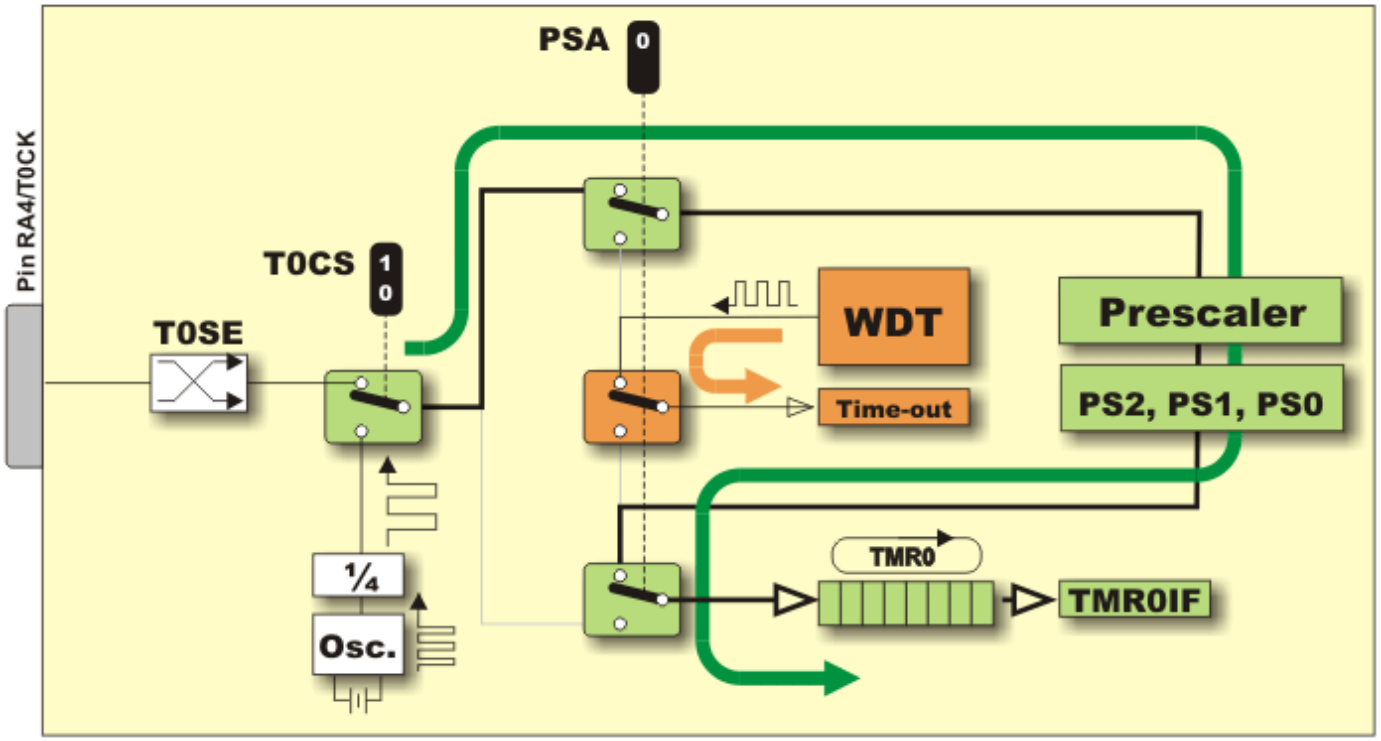
Tỷ lệ chia của prescaler được điều chỉnh bằng cách kết hợp ba bit này.
Lưu ý: cùng một tổ hợp bit sẽ cho tỷ lệ chia khác nhau đối với TMR0 và WDT.



OPTION_REG	R/W (1)	R/W (1)	R/W (1)	R/W (1)	R/W (1)	R/W (1)	R/W (1)	R/W (1)	Features
	RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

PS2	PS1	PS0	TMR0	WDT
0	0	0	1:2	1:1
0	0	1	1:4	1:2
0	1	0	1:8	1:4
0	1	1	1:16	1:8
1	0	0	1:32	1:16
1	0	1	1:64	1:32
1	1	0	1:128	1:64
1	1	1	1:256	1:128

Timer hoạt động như một bộ đếm



Bộ đếm sản phẩm hiển thị trên LED 7 đoạn/LCD.

```
#include <16F887.h>
#include <lcd.c> // Thu viện LCD của CCS
#fuses HS,NOWDT,NOPUT,NOLVP
#use delay(clock=8000000)
// Cấu hình chân LCD
........
//=====================================
void main() {
// Cấu hình Timer0 ở chế độ Counter
      setup_timer_0(T0_EXT_L_TO_H | T0_DIV_1);
// Đếm cạnh lên từ chân RA4 (T0CKI)
       set_timer0(0); // Khởi tạo giá trị bộ đếm
       lcd_init(); // Khởi tạo LCD
unsigned int8 count;
while(TRUE) {
    delay_ms(500);
    count = get_timer0(); // Đọc giá trị bộ đếm
    //htLed7Seg(count);
    lcd_gotoxy(1,1); // Dòng 1, cột 1
    printf(lcd_putc, "Counter: %u",count);
  }
}
```

Các lệnh tương tác với Timer trong CCS C

- Lệnh SETUP_TIMER_X()
- Lệnh SET_TIMER_X()
- Lệnh SETUP_COUNTERS()
- Lệnh SETUP_WDT()
- Lệnh RESTART_WDT()
- Lệnh GET_TIMER_X()

LỆNH SETUP_TIMER_0(MODE)

Cú pháp: setup_timer_0(mode)

Chức năng: Định cấu hình cho TIMER0.

Thông số: mode có thể là 1 hoặc 2 hằng số định nghĩa trong file device.h.

Các thông số gồm

RTCC_INTERNAL, RTCC_EXT_L_TO_H hoặc RTCC_EXT_H_TO_L

RTCC_DIV_2, RTCC_DIV_4, RTCC_DIV_8, RTCC_DIV_16, RTCC_DIV_32, RTCC_DIV_64, RTCC_DIV_128, RTCC_DIV_256.

Các hằng số từ nhiều nhóm khác nhau thì có thể or với nhau.

LỆNH SET_TIMERx(value)

Cú pháp: set_timerX(value) ; x là 0, 1, 2

Thông số: value là hằng số nguyên 8 hoặc 16 bit dùng để thiết lập giá trị mới cho timer.

Chức năng: thiết lập giá trị bắt đầu cho TIMER.

Có hiệu lực: cho tất cả các vi điều khiển PIC có timer.

LỆNH GET_TIMERx()

Cú pháp: value = get_timerX() ; x là 0, 1, 2

Thông số: không có.

Chức năng: đọc giá trị của TIMER/COUNTER.

Có hiệu lực: cho tất cả các vi điều khiển PIC có timer

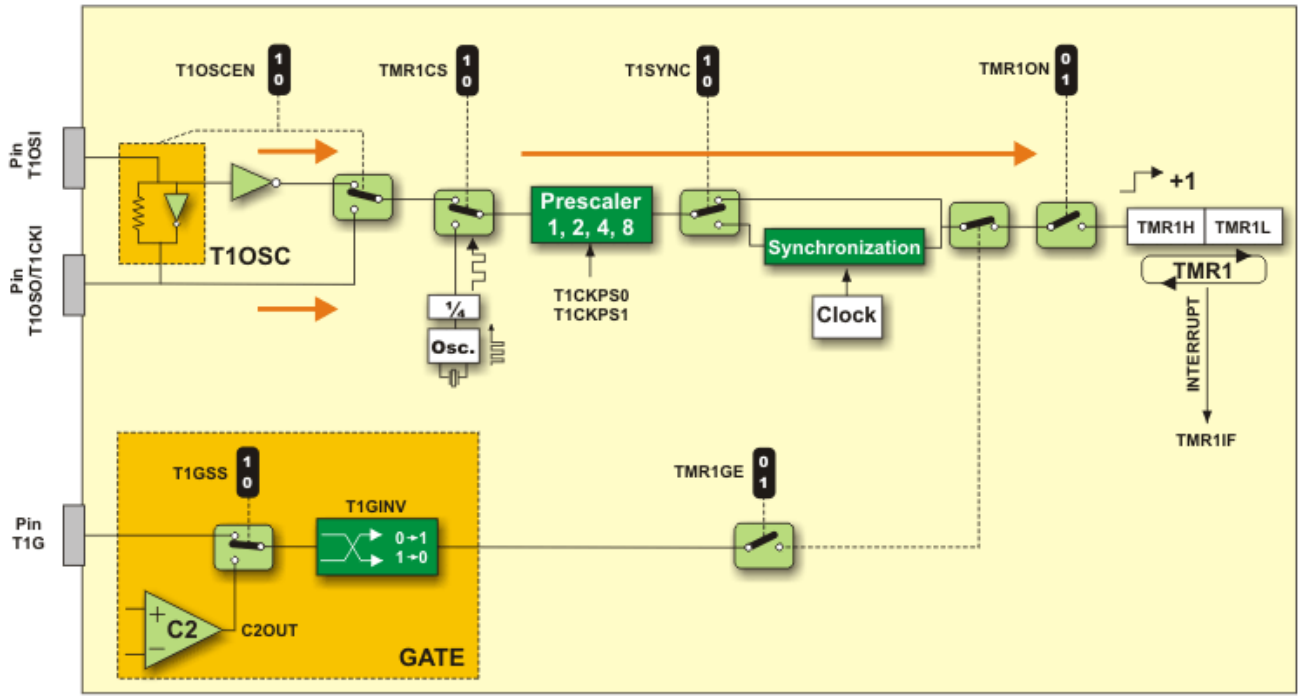
Các lệnh khác tham khảo tại sách trang 112

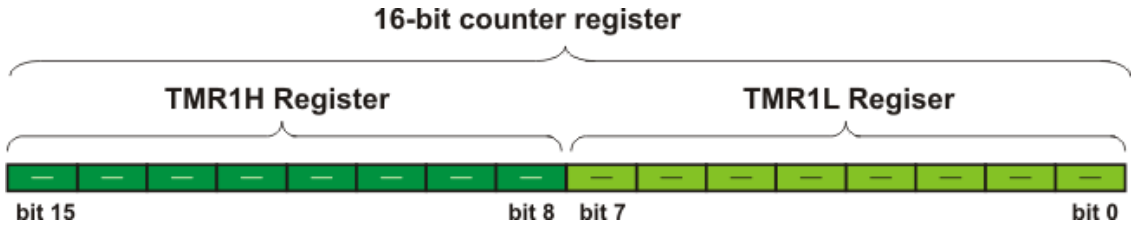
```
#include <16F887.h>
#fuses INTRC_IO, NOWDT, PUT, MCLR, NOPROTECT, NOCPD, BROWNOUT
#use delay(clock=8000000)

// Định nghĩa mã LED 7 đoạn (Anode chung)
unsigned int8 const ma7doan[] = {0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82,
0xF8, 0x80, 0x90};

unsigned int8 gio = 0, phut = 0, giay = 0;
unsigned int16 count_ms = 0;

#int_timer0
void interrupt_timer0() {
   set_timer0(6); // Nạp lại giá trị để ngắt mỗi 1ms
   count_ms++;

   if (count_ms >= 1000) {
      count_ms = 0;
      giay++;
      if (giay >= 60) { giay = 0; phut++; }
      if (phut >= 60) { phut = 0; gio++; }
      if (gio >= 24)  { gio = 0; }
   }
}
```

```
void hien_thi() {
--------------------------------
}
void main() {
   setup_oscillator(OSC_8MHZ);

   set_tris_d(0x00);
   set_tris_c(0x00);

   // Cấu hình Timer 0: Nguồn xung nội, bộ chia 8
   setup_timer_0(RTCC_INTERNAL | RTCC_DIV_8);
   set_timer0(6);

   enable_interrupts(INT_TIMER0);
   enable_interrupts(GLOBAL);

   while(TRUE) {
      hien_thi();
   }
}
```

$$T_{int} = \frac{4}{F_{osc}} \times Prescaler \times (256 - \mathrm{TMR0})$$

Timer 1





	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	Features
T1CON	T1GINV	TMR1GE	T1CKPS1	T1CKPS0	T1OSCEN	T1SYNC	TMR1CS	TMR1ON	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

Bit 7 T1GINV: bit đảo cổng của Timer1 - Timer1 Gate Invert bit

1= cổng Timer1 tích cực mức 1 (Timer1 đếm khi cổng ở mức 1).

0= cổng Timer1 tích cực mức 0 (Timer1 đếm khi cổng ở mức 0).

Bit 6 TMR1GE: bit cho phép cổng của Timer1 - Timer1 Gate Enable bit

Nếu TMR1ON = 0: bit này sẽ không có tác dụng.

Nếu TMR1ON = 1:

1= Timer1 mở nếu cổng của Timer1 không tích cực.

0= Timer1 mở.

Bit 5-4 T1CKPS1:T1CKPS0: các bit lựa chọn bộ chia - Timer1 input Clock Prescale Select bits

11=1:8 giá trị chia.

10=1:4 giá trị chia.

01=1:2 giá trị chia.

00=1:1 giá trị chia.

Bit 3 T1OSCEN: bit ĐK cho phép bộ dao động Timer1 - Timer1 Oscillator Enable Control bit

1= bộ dao động được phép.

0= Tắt bộ dao động.

Bit 2 T1SYNC: bit ĐK đồng bộ ngõ vào xung clock bên ngoài của timer1 Khi TMR1CS = 1:

1= không thể đồng bộ ngõ vào clock ở từ bên ngoài.

0= đồng bộ ngõ vào clock ở từ bên ngoài.

Khi TMR1CS = 0:

Bit này bị bỏ qua. Timer1 dùng xung clock bên trong khi TMR1CS = 0.

Bit 1 TMR1CS: bit lựa chọn nguồn xung clock của timer1

1= Chọn nguồn xung clock từ bên ngoài ở chân RC0/T1OSO/T1CKI (cạnh lên).

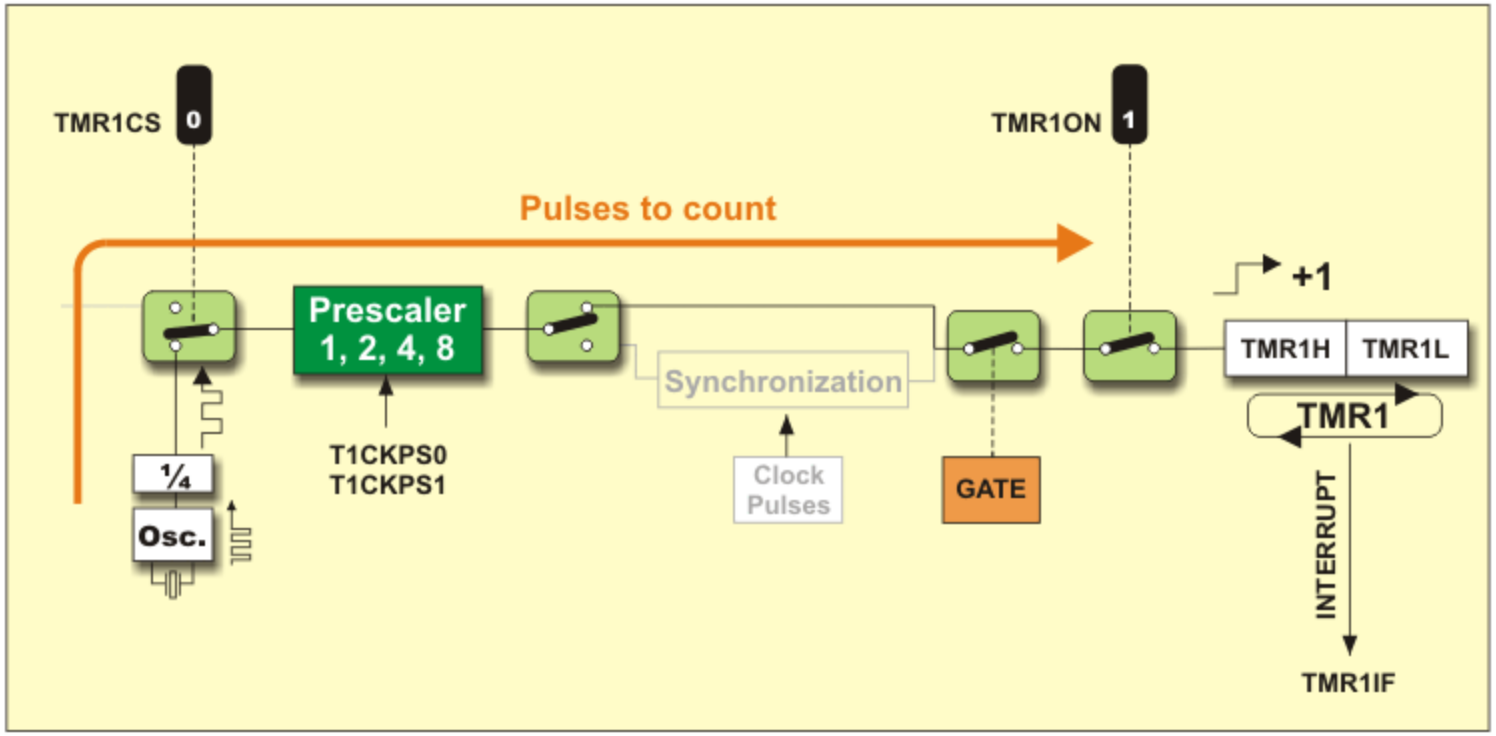
0= Chọn xung nội bên trong (FOSC/4).

Bit 0 TMR1ON: bit điều khiển Timer1

1= Cho phép Timer1 đếm.

0= Timer1 ngừng đếm

TMR1 IN TIMER MODE



LỆNH SETUP_TIMER_1(MODE)

Cú pháp: setup_timer_1(mode)

Thông số: mode có thể là 1 hoặc 2 hằng số định nghĩa trong file device.h. Các thông số gồm

T1_DISABLED, T1_INTERNAL, T1_EXTERNAL, T1_EXTERNAL_SYNC

TC_CLK_OUT, T1_DIV_BY_1, T1_DIV_BY_2, T1_DIV_BY_4, T1_DIV_BY_8.

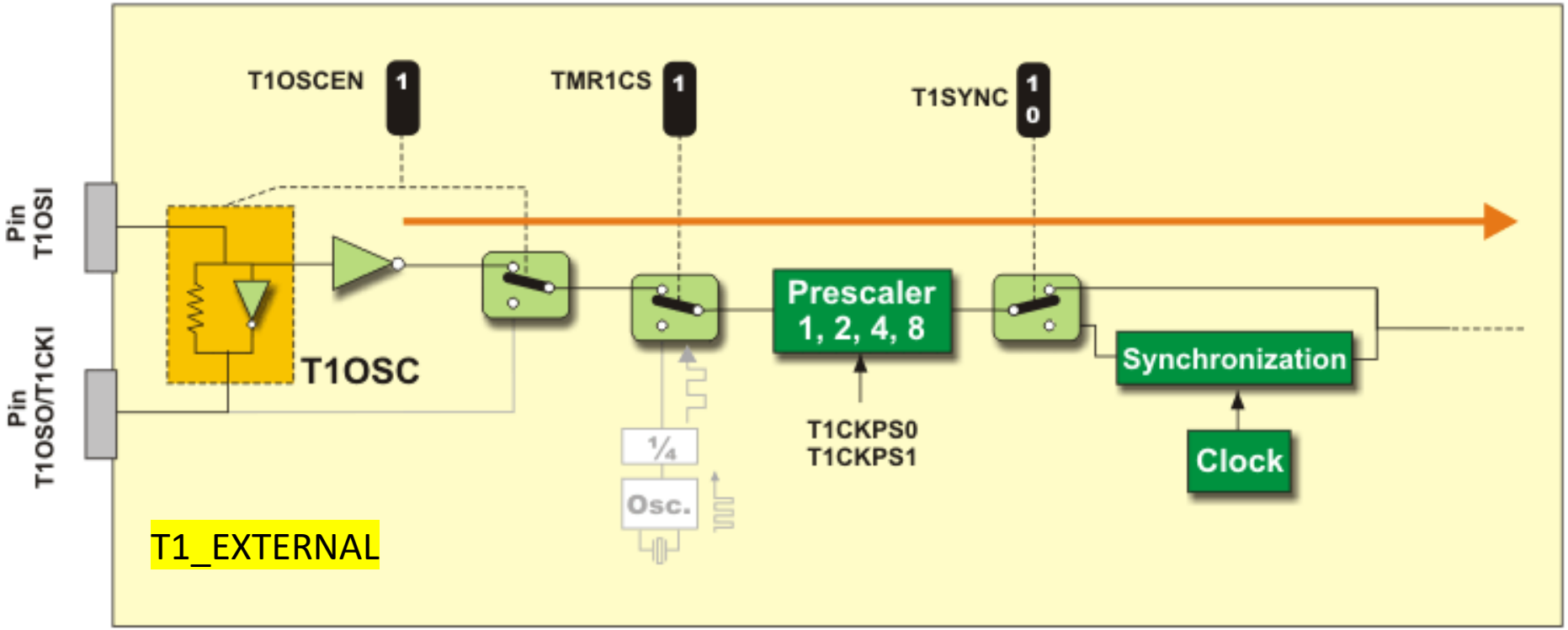
Các hằng số từ nhiều nhóm khác nhau thì có thể or với nhau.

Chức năng: Khởi tạo cho TIMER1.

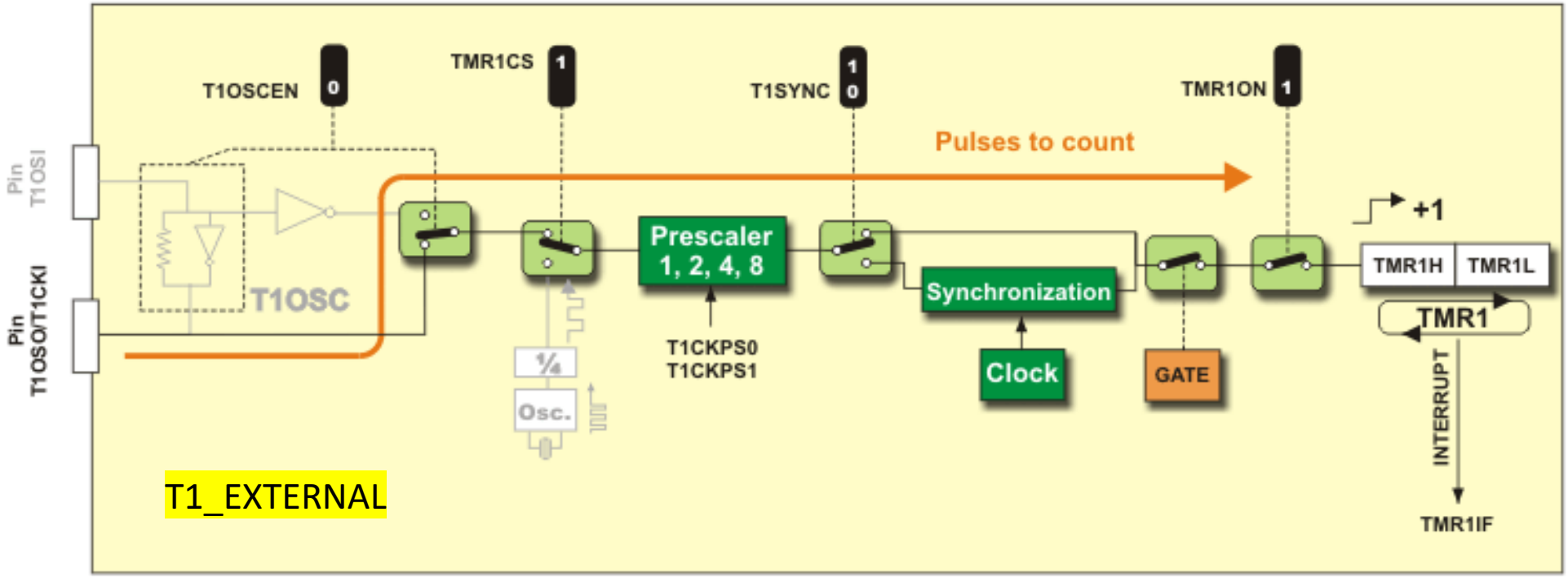
Với tần số thạch anh là 20MHz và khởi tạo T1_DIV_BY_8 thì timer sẽ tăng giá trị sau mỗi khoảng thời gian 1.6µs. Timer sẽ tràn sau 104.8576 ms.

Có hiệu lực: Cho tất cả các vi điều khiển PIC có timer 1.

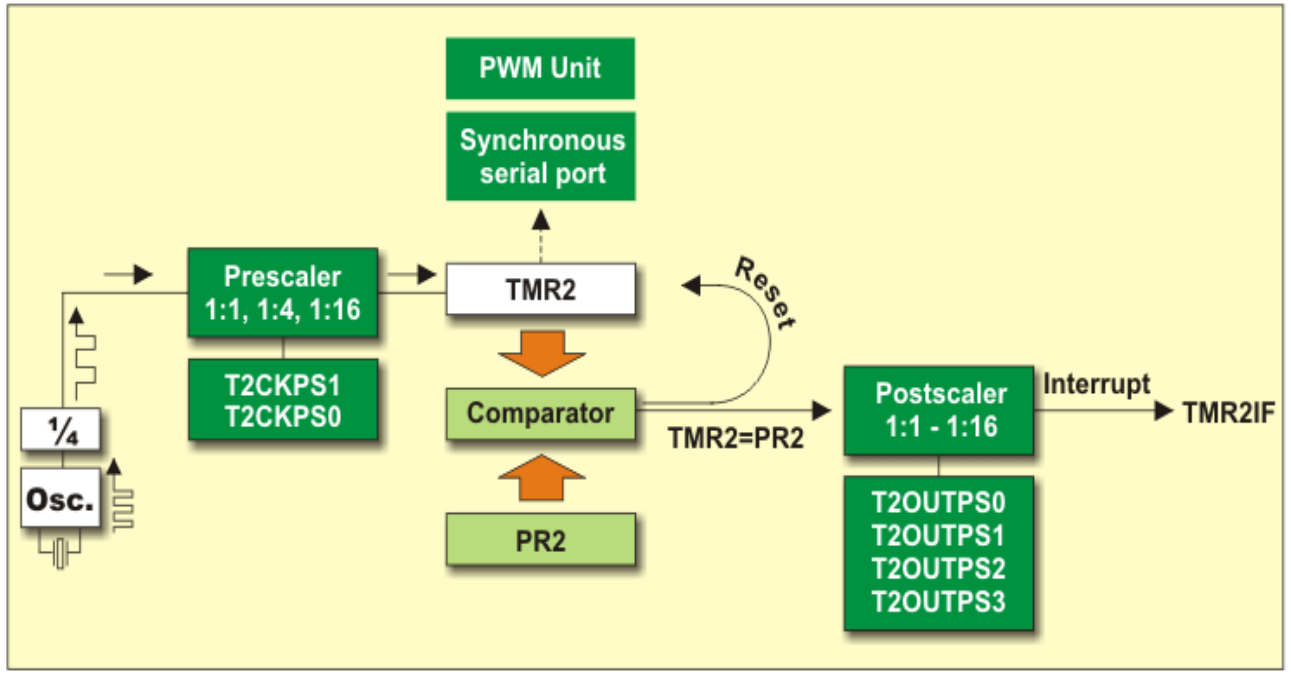
TMR1 IN TIMER MODE với external osc



TMR1 IN COUNTER MODE



Timer TMR2



Bit 7 Chưa sử dụng nếu đọc sẽ có giá trị „0".

Bit 6-3 TOUTPS3:TOUTPS0: các bit lựa chọn ngõ ra bộ chia sau (Postscaler) của Timer2

0000 =1:1 0001 =1:2 0010= 1:3 0011=1:4
0100 =1:5 0101 =1:6 0110= 1:7 0111=1:8
1000 =1:9 1001 =1:10 1010= 1:11 1011=1:12
1100 =1:13 1101 =1:14 1110= 1:15 1111=1:16

Bit 2 TMR2ON: Bit điều khiển cho phép/cấm Timer2

1= cho phép timer2 đếm.
0= Timer2 ngừng đếm.

Bit 1-0 T2CKPS1:T2CKPS0: bit lựa chọn hệ số chia trước cho nguồn xung clock của timer2

00= hệ số chia là 1.
01= hệ số chia là 4.
1x= hệ số chia là 16.

		R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	Features
T2CON	-	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

Các lệnh cấu hình cho Timer2

LỆNH SETUP_TIMER_2(MODE)

Cú pháp: setup_timer_2(mode, period, postscale)

Thông số: mode có thể là 1 trong các thông số: T2_DISABLED, T2_DIV_BY_1, T2_DIV_BY_4, T2_DIV_BY_16

Period là số nguyên có giá trị từ 0 đến 255 dùng để xác định khi nào timer2 bị reset hay giá này trị dùng để nạp cho thanh ghi PR2.

postscale là số nguyên có giá trị từ 1 đến 16 dùng để xác định timer **tràn bao nhiêu lần** trước khi phát sinh tín hiệu ngắt – chọn tỉ lệ chia của bộ chia sau.

Chức năng: khởi tạo cho TIMER2.

Mode chỉ định kiểu bộ chia của timer từ tần số của mạch dao động.

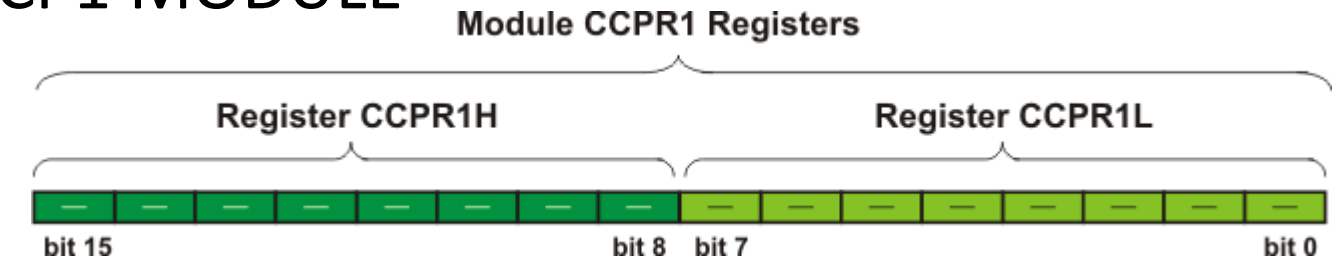
Giá trị của timer có thể đọc hoặc ghi dùng lệnh GET_TIMER2() và SET_TIMER2().

TIMER2 là timer 8 bit.

Có hiệu lực: cho tất cả các vi điều khiển PIC có timer 2.

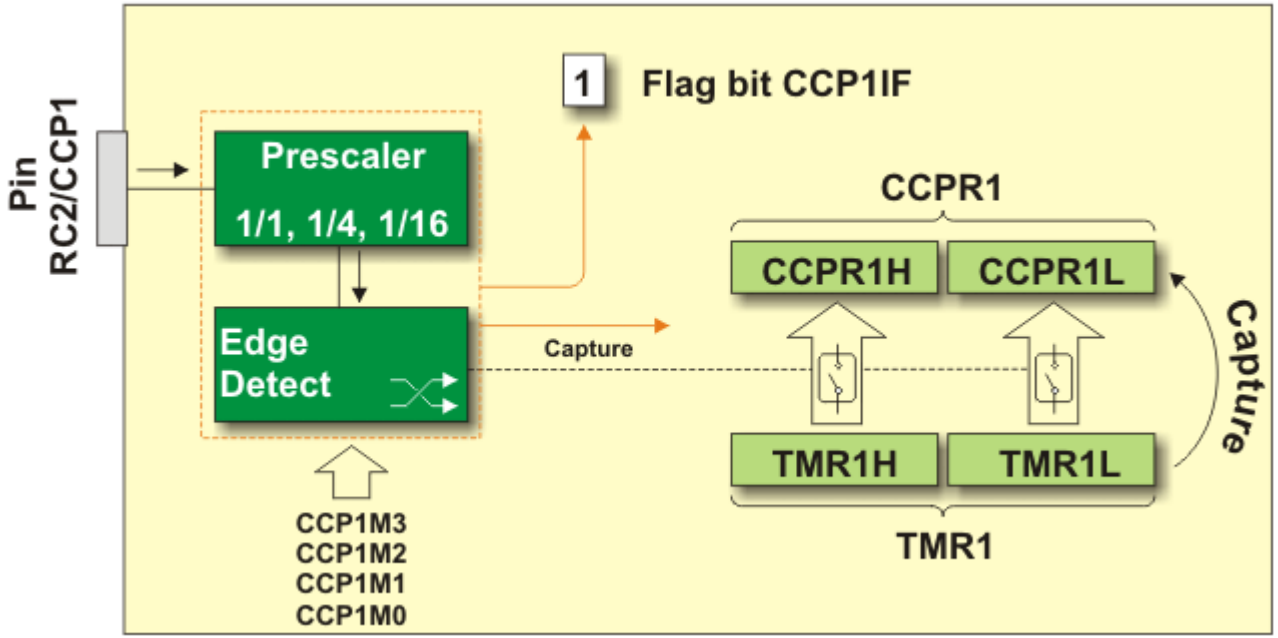
Chế độ CCP module (Capture/compare/PWM)

- CCP1 MODULE




```
#include <16F887.h>
#fuses HS,NOWDT,NOPROTECT,NOLVP
#use delay(clock=20000000)   // Thạch anh 20MHz
volatile unsigned int16 capture_value = 0;
volatile unsigned int16 last_capture = 0;
volatile unsigned int16 period = 0;
```

CCP1 IN CAPTURE MODE




```
#INT_CCP1                    // Ngắt CCP1
void ccp1_isr() {
  capture_value = CCP_1; // Đọc giá trị capture
  period = capture_value - last_capture;
   // Tính chu kỳ   last_capture = capture_value;
}
void main() {
  // Cấu hình Timer1
  setup_timer_1(T1_INTERNAL | T1_DIV_BY_1);
  // Timer1 chạy với Fosc/4
  // Cấu hình CCP1 ở chế độ Capture, bắt cạnh lên
setup_ccp1(CCP_CAPTURE_RE);
  // Cho phép ngắt CCP1
  enable_interrupts(INT_CCP1);
  enable_interrupts(GLOBAL);
  while(TRUE) {
  // Ví dụ: xuất chu kỳ đo được ra UART
    printf("Chu ky: %lu\n", period);
    delay_ms(500);
  }
}
```

Các lệnh sử dụng để cấu hình CCP1

Lệnh	Chức năng
setup_ccp1(mode)	Cấu hình CCP1 theo chế độ mong muốn: Capture, Compare hoặc PWM.
CCP_1	Thanh ghi chứa giá trị capture (dùng trong ISR khi có sự kiện capture).
set_pwm1_duty(value)	Đặt giá trị duty cycle cho CCP1 khi ở chế độ PWM.
setup_timer_2(divisor, period, p)	Cấu hình Timer2 (thường dùng cho PWM).
setup_timer_1(options)	Cấu hình Timer1 (thường dùng cho Capture/Compare).
#INT_CCP1	Khai báo hàm ngắt CCP1.
enable_interrupts(INT_CCP1)	Cho phép ngắt CCP1.

Các chế độ cấu hình với setup_ccp1(mode)

- **Capture mode:**

CCP_CAPTURE_RE→ Bắt cạnh lên (Rising Edge).
CCP_CAPTURE_FE→ Bắt cạnh xuống (Falling Edge).
CCP_CAPTURE_DIV_4→ Bắt cạnh lên mỗi 4 lần.
CCP_CAPTURE_DIV_16→ Bắt cạnh lên mỗi 16 lần.

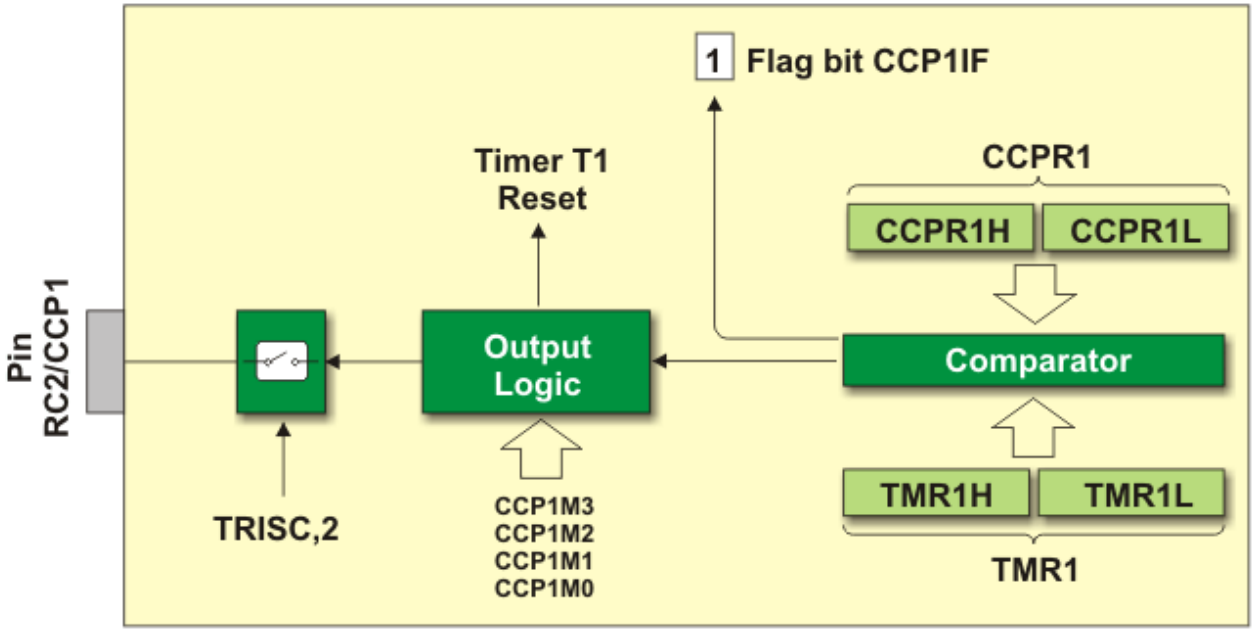
- **Compare mode:**

CCP_COMPARE_SET_ON_MATCH
→ Đặt mức logic khi trùng khớp.
CCP_COMPARE_CLR_ON_MATCH
→ Xóa mức logic khi trùng khớp.
CCP_COMPARE_TOGGLE_ON_MATCH
→ Đảo trạng thái khi trùng khớp.
CCP_COMPARE_INIT_HIGH
→ Khởi tạo mức cao.
CCP_COMPARE_INIT_LOW
→ Khởi tạo mức thấp.

- **PWM mode:**

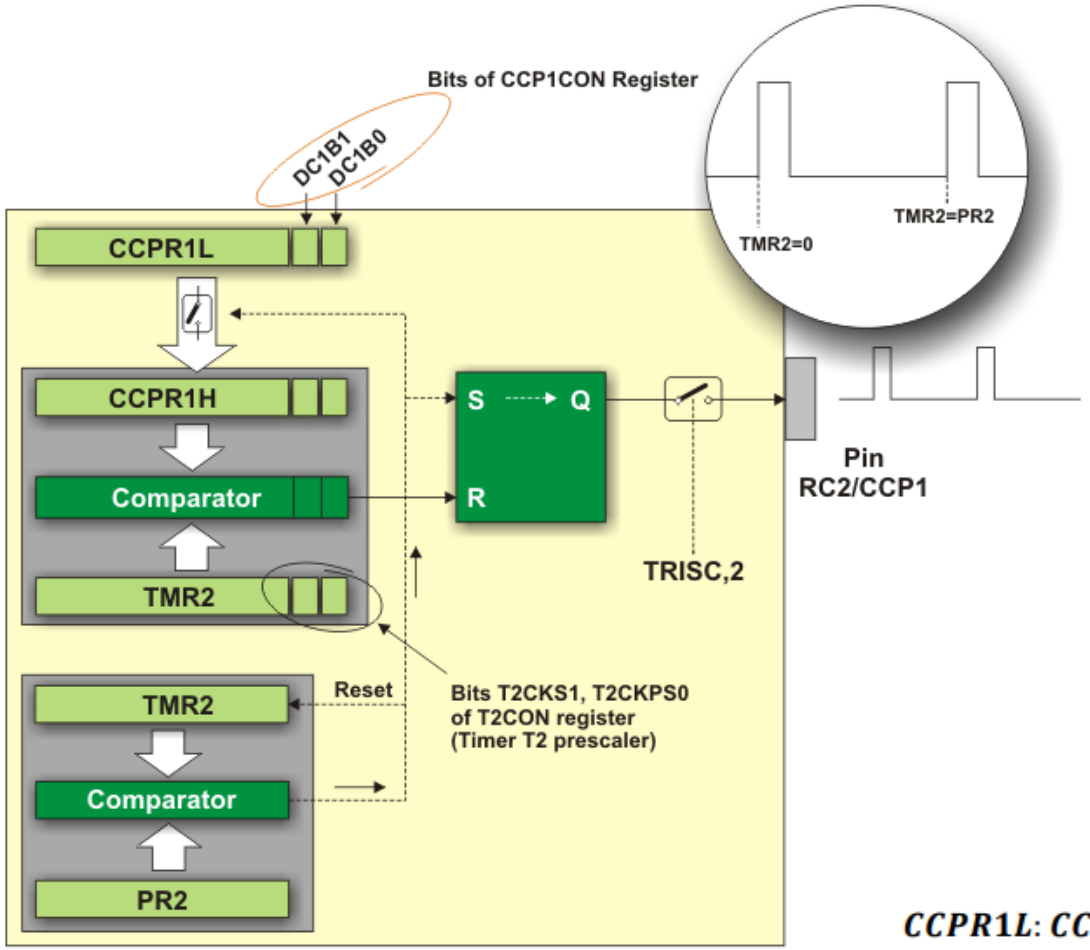
CCP_PWM → Cấu hình CCP1 ở chế độ PWM.

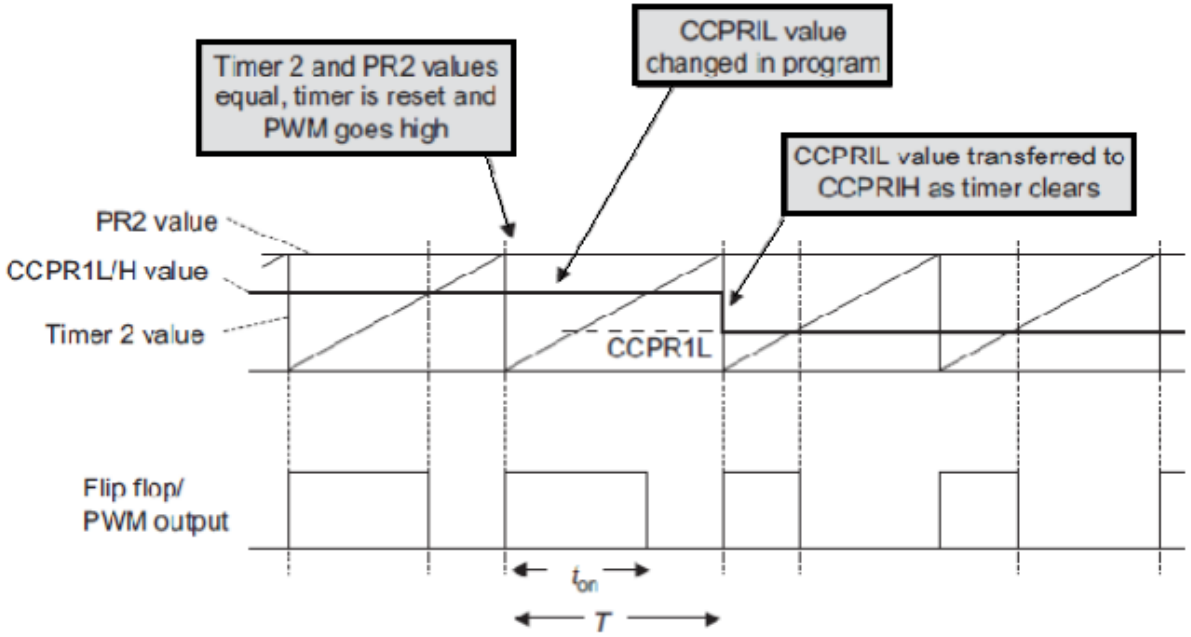
CCP1 IN COMPARE MODE




```
#include <16F887.h>
#fuses HS,NOWDT,NOPROTECT,NOLVP
#use delay(clock=20000000)          // Thạch anh 20MHz
#define COMPARE_VALUE 50000   // Giá trị so sánh
#INT_CCP1
void ccp1_isr() {                   // Ngắt CCP1
// Khi Timer1 trùng khớp với COMPARE_VALUE, CCP1 sẽ đảo trạng thái
   output_toggle(PIN_C0);     // Ví dụ: đảo trạng thái chân RC0
}
void main() {                       // Cấu hình Timer1
   setup_timer_1(T1_INTERNAL | T1_DIV_BY_1);
                                    // Timer1 chạy với Fosc/4
  // Cấu hình CCP1 ở chế độ Compare, đảo trạng thái khi trùng khớp
   setup_ccp1(CCP_COMPARE_TOGGLE_ON_MATCH);
   set_ccp1(COMPARE_VALUE);       // Nạp giá trị so sánh
   enable_interrupts(INT_CCP1);      // Cho phép ngắt CCP1
   enable_interrupts(GLOBAL);

  while(TRUE) {
   // Vòng lặp chính có thể thực hiện các tác vụ khác
   delay_ms(100);
  }
}
```

CCP1 in PWM mode





PWM Period = T_{pwm} = (PR2 + 1) x (Timer 2 input Clock)

PWM Period = T_{pwm} = (PR2 + 1) x (4 * Tosc * TMR2 Prescaler value)

Therefore to find the PR2 for desired PWM Period

$$PR2 = \left\{\left(\frac{\text{Tpwm}}{4 * Tosc * (TMR2\ Prescaler\ Value)}\right) - 1\right\}$$

Above equation can be represented in terms of frequency as below.

$$PR2 = \left\{\left(\frac{Fosc}{4 * Fpwm * (TMR2\ Prescaler\ Value)}\right) - 1\right\}$$

$$CCPR1L{:}\,CCP1CON < 5{:}4 > = \left\{\left(\frac{Ton}{Tosc * (TMR2\ Prescaler\ Value)}\right)\right\}$$

Above equation can be represented in terms of frequency and percentage Duty cycle (%DCpwm) as below.

$$CCPR1L{:}\,CCP1CON < 5{:}4 > = \left\{\left(\frac{\%DCpwm * Fosc}{100 * Fpwm * (TMR2\ Prescaler\ Value)}\right)\right\}$$

Vi dụ

```
#include <16F877A.h>
#fuses HS,NOWDT,NOPROTECT,NOLVP
#use delay(clock=8000000)  // 8 MHz
void main() {
        setup_timer_2(T2_DIV_BY_4, 124, 1);   // Prescaler=4, PR2=124 → ~500Hz PWM
        setup_ccp1(CCP_PWM);                  // Cấu hình CCP1 ở chế độ PWM
        set_pwm1_duty(250);                   // Duty cycle ban đầu ~50% (250/500)
  while(TRUE) {
    // Ví dụ: tăng giảm duty cycle theo thời gian
        for(int duty=0; duty<500; duty+=10) {
                set_pwm1_duty(duty);
                delay_ms(50);
        }
    for(int duty=500; duty>0; duty-=10) {
      set_pwm1_duty(duty);
      delay_ms(50);
    }
  }
}
```